



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 2

по курсу «Языки и методы программирования»

«Класс множества точек с вычислением минимальной площади
прямоугольника»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Индивидуальный вариант	1
Реализация	1
Класс Point	1
Класс PSet	2
HTTP-сервер	4
Протокол тестирования	7
Тест 1: Три точки	8
Тест 2: Пять точек	8
Тест 3: Точки с отрицательными координатами	8
Вывод	8

Цель работы

Целью данной работы является изучение базовых возможностей языка Java.

Индивидуальный вариант

Вариант 21: Класс, представляющий конечное множество точек на плоскости с операцией вычисления минимальной площади прямоугольника, содержащего все точки (любая сторона прямоугольника параллельна одной из осей координат).

Дополнительно выполнена обёртка класса в веб-сервер (+2 балла).

Реализация

Созданы три файла: Point.java, PSet.java и Test.java (веб-сервер).

Класс Point

Класс Point представляет точку на плоскости с координатами x и y.

Point.java

```
public class Point {
    private double x, y;

    public Point(double x, double y) {
        this.x = x;
        this.y = y;
    }

    public double getX() { return x; }
    public double getY() { return y; }

    public String toString() {
        return "("+x+", "+y+"";
    }
}
```

Класс PSet

Класс PSet представляет множество точек на плоскости и содержит метод `getMinArea()`, который вычисляет минимальную площадь прямоугольника, параллельного осям координат, содержащего все точки множества. Площадь вычисляется как произведение ширины и высоты: $S = (\max X - \min X) \times (\max Y - \min Y)$.

```
public class PSet {
    private Point[] point_set;

    public PSet(Point[] arr) {
        point_set = arr;
    }

    public double getMinArea() {
        if (point_set.length == 0) {
            return 0.0;
        }

        double minX = point_set[0].getX();
        double maxX = point_set[0].getX();
        double maxY = point_set[0].getY();
        double minY = point_set[0].getY();

        for (int i = 0; i < point_set.length; i++) {
            Point p = point_set[i];
            double x = p.getX();
            double y = p.getY();

            if (x < minX) minX = x;
            if (x > maxX) maxX = x;
            if (y < minY) minY = y;
            if (y > maxY) maxY = y;
        }

        double width = maxX - minX;
        double height = maxY - minY;

        return width * height;
    }

    public String toString() {
        if (point_set.length == 0) {
            return "{}";
        }
        String result = "{";
        for (int i = 0; i < point_set.length; i++) {
            result += point_set[i];
            if (i < point_set.length - 1) {
                result += ", ";
            }
        }
        result += "}";
        return result;
    }
}
```

HTTP-сервер

Сервер реализован в файле `Test.java` с использованием встроенного класса `com.sun.net.httpserver.HttpServer`. Сервер:

- Слушает порт 7557;
- Обрабатывает GET-запросы;
- Принимает параметр `points` в формате `x1,y1;x2,y2; ... ;xn,yn`;
- Возвращает JSON-ответ с вычисленной площадью и списком точек.

Test.java

```
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.net.URLDecoder;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

class QueryParser {

    static Map<String, String> parse(String query) {
        Map<String, String> params = new HashMap<>();
        if (query == null || query.isEmpty())
            return params;

        for (String pair : query.split("&")) {
            String[] parts = pair.split("=", 2);
            if (parts.length != 2)
                continue;

            String key = URLDecoder.decode(parts[0],
StandardCharsets.UTF_8);
            String value = URLDecoder.decode(parts[1],
StandardCharsets.UTF_8);
            params.put(key, value);
        }
        return params;
    }
}

class PSetMapper {
    static PSet fromParams(Map<String, String> params) {
        String pointsStr = params.get("points");

        String[] tokens = pointsStr.split(";");
        Point[] points = new Point[tokens.length];

        for (int i = 0; i < tokens.length; i++) {
            String[] coords = tokens[i].split(",");
            double x = Double.parseDouble(coords[0].trim());
            double y = Double.parseDouble(coords[1].trim());
            points[i] = new Point(x, y);
        }
        return new PSet(points);
    }
}
```

Test.java

```
class PSetJsonSerializer {
    static String toJson(PSet pset) {
        double area = pset.getMinArea();
        String pointsStr = pset.toString();

        String escapedPoints = pointsStr.replace("\\", "\\");

        return ""
            {
                "area": %s,
                "points": "%s"
            }
            "".formatted(area, escapedPoints);
    }
}
```

Test.java (продолжение)

```
public class Test {
    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(new
        InetSocketAddress(7557), 0);
        server.createContext("/", new MyHandler());
        server.setExecutor(null); // creates a default executor
        server.start();
    }

    static class MyHandler implements Handler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            if (!"GET".equals(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(405, -1);
                return;
            }

            String query = exchange.getRequestURI().getQuery();
            Map<String, String> params = QueryParser.parse(query);

            PSet pset;
            pset = PSetMapper.fromParams(params);

            String json = PSetJsonSerializer.toJson(pset);

            exchange.getResponseHeaders().add("Content-Type",
            "application/json");
            exchange.sendResponseHeaders(200, json.getBytes().length);

            try (OutputStream os = exchange.getResponseBody()) {
                os.write(json.getBytes());
            }
        }
    }
}
```

Протокол тестирования

Запуск сервера производится командой:

```
server@net1.yss.su
```

```
java Test.java
```

Тестирование выполняется с помощью утилиты curl.

Тест 1: Три точки

arseny@local-pc

```
$ curl 'http://net1.yss.su:7557/?points=0.0,0.0;1.1,1.1;2.0,3.3'
{
  "area": 6.6,
  "points": "{(0.0, 0.0), (1.1, 1.1), (2.0, 3.3)}"
}
```

Площадь вычисляется как $(2.0 - 0.0) \times (3.3 - 0.0) = 2.0 \times 3.3 = 6.6$.

Тест 2: Пять точек

arseny@local-pc

```
$ curl 'http://net1.yss.su:7557/?points=0.0,0.0;1.1,1.1;2.0,3.3;5.2,3.2;1.6,2.2'
{
  "area": 17.16,
  "points": "{(0.0, 0.0), (1.1, 1.1), (2.0, 3.3), (5.2, 3.2), (1.6, 2.2)}"
}
```

Площадь: $(5.2 - 0.0) \times (3.3 - 0.0) = 5.2 \times 3.3 = 17.16$.

Тест 3: Точки с отрицательными координатами

arseny@local-pc

```
$ curl 'http://net1.yss.su:7557/?points=0.0,0.0;1.1,1.1;2.0,3.3;5.2,-3.2;1.6,-2.2'
{
  "area": 33.800000000000004,
  "points": "{(0.0, 0.0), (1.1, 1.1), (2.0, 3.3), (5.2, -3.2), (1.6, -2.2)}"
}
```

Площадь: $(5.2 - 0.0) \times (3.3 - (-3.2)) = 5.2 \times 6.5 = 33.8$.

Вывод

В ходе лабораторной работы был реализован класс Point для представления точки на плоскости и класс PSet для работы с множеством точек. Класс PSet предоставляет метод getMinArea(), который вычисляет минимальную площадь прямоугольника, параллельного осям координат и содержащего все точки множества.

Созданный HTTP-сервер успешно обрабатывает GET-запросы, парсит параметры координат точек и возвращает результаты вычислений в формате JSON.

Тестирование подтвердило корректность работы алгоритма вычисления площади для различных наборов точек, включая точки с отрицательными координатами.